CampusInbox: An Evidence-Gated Agent for College Deadline Management

Kareena Jahagirdar
Department of Computer Engineering, Pimpri Chinchwad College of Engineering, Pune, India
PRN 123B1B048, Division A kareena.jahagirdar23@pccoepune.org

***Abstract*-**College students miss deadlines because the dates arrive scattered across course mails and learning-platform notifications, and the tools they already have either resurface whole mails without reading them or act on extracted dates without showing any proof. This paper presents CampusInbox, a local agent-style application that turns college mail into reviewed deadlines, reminders, calendar entries and draft replies to faculty. Its design follows one rule: evidence before action. A dedicated EvidenceExtractor role proposes a deadline only together with a verbatim quote from the source mail, every extracted date waits for one-tap human approval, and the application has no mail-send capability at all. We evaluate the running system on a purpose-built, clearly synthetic corpus of 65 college-style mails in six categories. The evidence-gated extractor auto-extracted 25 deadlines with 100% precision, guessed a date on 0 of 20 ambiguous or deadline-free mails, and produced zero wrong auto-extracted dates overall, against 26 for a naive first-date baseline on the same corpus. A 23-test unit suite and an 8-step end-to-end walkthrough pass in full. The results show that a small, deterministic, evidence-gated core can safely carry the parts of deadline management where a mistake costs the most.

***Keywords*-**agentic systems, deadline extraction, human-in-the-loop, prompt injection, evidence grounding, personal productivity agents

I. INTRODUCTION

A final-year engineering student receives deadline-bearing mail from several courses, a learning platform that posts its own notifications, and ad-hoc announcements that quietly change earlier dates. In our setting the failure mode is ordinary and expensive: a lab report due "next Friday" is read once, forgotten, and discovered after the deadline. The tools a student already has do not close this loop. Google Classroom lists assigned and missing work but does not read mail [10]. Gmail nudges resurface whole messages without extracting what is due or when [11]. Commercial inbox assistants such as SaneBox add reminders and snoozing but require handing a third party ongoing access to the mailbox [12].

A natural response is to point a large language model at the inbox and let it act. Recent agent frameworks make this easy to build [1], [3]. It is also where a new failure mode appears: the model reads hostile text as instructions, invents a plausible date, or fires an action the student never approved [8], [9]. For a deadline tracker, one invented date is worse than no extraction at all.

CampusInbox starts from the opposite end. It is a small, local, agent-style application built around a single rule: evidence before action. A dedicated EvidenceExtractor role may propose a deadline only with a verbatim quote from the source mail. A human review gate approves every date in one tap; nothing is reminded, exported or drafted before that. An ActionPlanner role assigns conservative state labels and writes draft replies to faculty that the application itself can never send, because no send endpoint exists. Everything else is deterministic code with tests.

The contributions of this paper are: (1) an architecture that splits an agent into two bounded reasoning roles surrounded by deterministic guardrails, including a review gate and a quote-verified model adapter; (2) a working local MVP implementing the full loop from mail import to reminders, calendar export and draft replies, with deadline revision handling; and (3) a measured evaluation of extraction accuracy and decision safety on a purpose-built synthetic corpus, compared against a naive baseline, with all test artifacts shipped for reproduction.

II. RELATED WORK

LLM agents that reason and act in loops were popularized by ReAct [1] and surveyed broadly in [2]; AutoGen structures them as conversations between roles [3]. CampusInbox borrows the role decomposition but keeps the roles narrow: each produces a proposal, and ordinary code decides what may happen to it. Temporal expression extraction is a mature rule-and-model problem [4], [5]; our offline baseline deliberately uses a far stricter grammar than these general taggers, trading recall for zero fabrication. Retrieval-augmented generation grounds answers in fetched text [6]; our quote check applies the same instinct to extraction, requiring the returned date to appear verbatim in the source. Human-AI interaction guidelines recommend showing users why a system did something and letting them correct it [7], which is exactly what the review gate operationalizes. On the threat side, indirect prompt injection shows that content read by an agent can carry instructions [8], and prompt injection tops the OWASP LLM risk list [9]; CampusInbox treats all mail as data and gives the reasoning roles no tools. Among products, Classroom [10], Gmail nudges [11] and SaneBox [12] each cover part of the workflow; Section VII compares them with CampusInbox feature by feature. The Enron corpus [13] is the standard public email collection and is relevant to future real-mail evaluation.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the architecture. Mail enters only through explicit imports (a JSON batch or a plain-text .eml file); the MVP has no live mailbox connection by design. An import and validation layer enforces size limits, deduplicates message IDs, rejects conflicting content under a repeated ID, and applies batches atomically. The EvidenceExtractor role then proposes course, task and a candidate deadline. In offline mode the extractor accepts only an explicit ISO 8601 timestamp with a timezone

offset on a labelled Due: line. Anything else stays undated instead of being guessed.

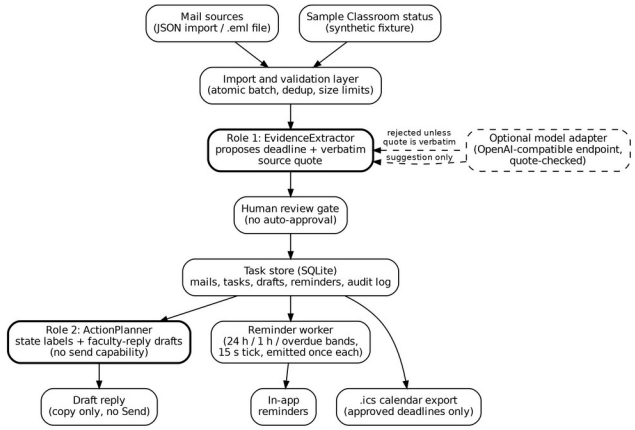


Fig. 1. CampusInbox architecture. Untrusted mail flows down the left; the optional model adapter may only suggest, and its suggestion is rejected unless the quote is verbatim.

An optional model adapter can replace the rules with an OpenAI-compatible endpoint chosen by the user, but a model proposal is accepted only if it carries a quote that appears verbatim in the mail and itself contains the proposed date; otherwise the import fails closed. This keeps a model useful where it is strong (reading free-form mail) while keeping its known weaknesses out of the decision path. The model mode was not exercised against a live provider in this work and we report no live-model numbers.

Downstream, a human review gate holds every extracted date until the student approves it against the shown quote and source. Only approved tasks feed the reminder worker, which emits each of three bands (24 hours before, 1 hour before, due/overdue) exactly once per task version on a 15-second tick. The ActionPlanner assigns state labels, including "Overdue - submission unverified" when the only evidence is the clock, and "Missing (sample Classroom evidence)" only when a synthetic status fixture says NOT_SUBMITTED; the label itself records that the evidence is synthetic. All state lives in a local SQLite database with an append-only audit log. Fig. 2 shows the task lifecycle, including revision handling: a new mail carrying an existing assignment key creates version n+1, returns the task to the review gate and deletes reminders that were tied to the superseded date.

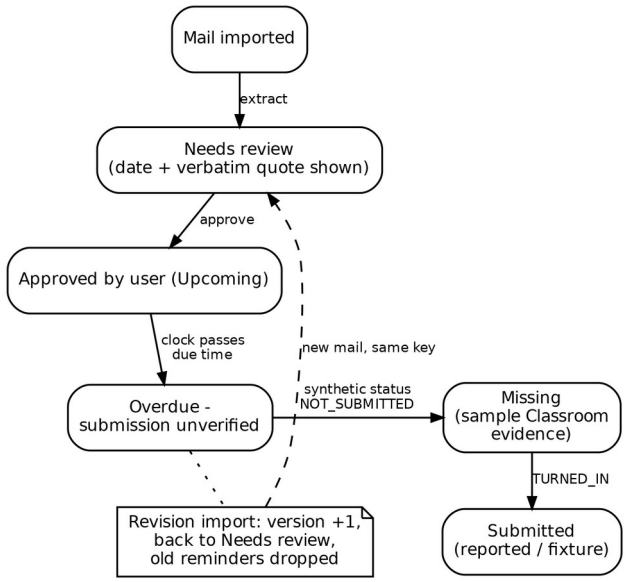


Fig. 2. Task lifecycle. Approval, revision and submission all change state through explicit events, never through model inference.

IV. METHODOLOGY

A. Evidence-gated extraction

The extractor returns a deadline only when it can point at the exact characters it read. In rules mode the match grammar is intentionally narrow: labelled Course:, Task: and Due: lines, with the date required to be a full timezone-aware ISO timestamp. Free-form dates, relative phrases and undated mails are routed to review with a clear reason instead of a guess. The same gate wraps the optional model mode: the response must be JSON, must contain due_at and quote, the quote must be a substring of the mail, and the date must appear inside the quote.

B. Review before reminders

A deadline that has not been approved produces no reminder, no calendar entry and no draft trigger. Approving is one tap after reading the quote; correcting means typing a verified date in ISO form. This follows [7]: the system shows its evidence and the user keeps the final say, at the cost of one click per task.

C. Draft-only faculty replies

The planner writes short, honest requests for submission instructions. It does not invent excuses, teacher names or promises, and it signs with the student's own name, PRN and division. The application exposes no send capability: drafts can be edited, saved and copied out, and that is all. Removing the send path removes the entire class of "agent mailed someone the wrong thing" failures.

D. Safety posture

Mail content is treated as data throughout; instruction-shaped text in a mail cannot reach any tool because the roles have none. The local server binds to loopback, validates the Host header, requires a per-run request token on writes, inserts all mail text as text rather than HTML, and applies a restrictive content-security policy. These are demo-grade protections, not a production audit, and the README says so.

V. IMPLEMENTATION

The MVP is a single-file Python 3.10 application (about 560 lines) using only the standard library: http.server for the loopback API, sqlite3 for persistence, and a background thread for the reminder tick. The UI is a plain black-and-white web page (HTML, CSS, vanilla JS) served by the same process, responsive down to phone widths. No pip install, API key or account is needed for the default demo. Five synthetic sample mails exercise the main paths: an overdue lab, a next-day report, a deadline 45 minutes out, an ambiguous "next Friday" date, and a mail containing hostile instruction text. A sixth mail demonstrates revision handling. Calendar export writes a standard .ics file containing approved deadlines only.

Testing is part of the deliverable: 23 offline unit tests cover extraction, the review gate, reminder bands, deduplication, revision invalidation, draft persistence, calendar correctness and hostile-content handling, including three tests where a mock model adapter tries to smuggle invented evidence and is rejected. An end-to-end script drives the real HTTP API through eight checks, including a process restart mid-run. A scripted Chrome walkthrough verified the rendered UI and captured the screenshots used in this paper. All artifacts ship with the code.

VI. EVALUATION

A. Dataset

There is no public corpus of college deadline mails, and real student inboxes contain private data that does not belong in a published dataset. We therefore built a clearly synthetic corpus of 65 author-written mails in six categories that mirror how college deadline mail actually varies (Table I). Every mail carries a ground-truth label: the exact deadline timestamp where one exists and is verifiable, or none. The corpus, its generator and the scoring script ship with the code so the numbers below can be reproduced byte for byte. We flag clearly: this corpus measures decision safety and behaviour on the distribution the app was built for. It does not measure accuracy on real inboxes; that remains future work and is discussed in Section IX.

TABLE I
SYNTHETIC EVALUATION CORPUS (65 MAILS)

Category	n	Ground truth
A. Structured Course/Task/Due format	15	Exact ISO timestamp
B. Free-form explicit deadline	20	Exact timestamp, varied phrasing
C. Ambiguous relative date	10	No verifiable date
D. No deadline at all	10	No date
E. Hostile instruction text	5	Exact timestamp; injection is data
F. Multiple dates, one authoritative	5	The labelled Due: line

B. Metrics and baseline

We report auto-extraction precision, recall over the 45 mails that contain a real deadline, and F1, with exact timestamp match (date and time, timezone-normalized) as the correctness criterion. We also report decision safety: the share of all mails on which the system did not auto-extract a wrong date. A wrong date silently accepted is the costly failure; an abstention costs one review tap. As a baseline we use a naive first-date extractor that parses the first date-like string it finds and guesses a concrete date for relative phrases, which is how a careless mail-reading agent behaves.

C. Results

TABLE II
EXTRACTION RESULTS ON THE 65-MAIL CORPUS (MEASURED RUNS)

Metric	Naive baseline	CampusInbox
Mails auto-extracted	52	25
Exactly correct	26	25
Wrong auto-extraction	26	0
Precision	50.0%	100%
Recall (45 deadline mails)	57.8%	55.6%
F1	53.6%	71.4%
Fabricated on ambiguous/undated (n=20)	10	0
Decision safety (no wrong auto-extracted date)	60.0%	100%

Table II and Fig. 3 show the trade the design makes. The naive baseline extracts more often and is wrong 26 times out of 65: it guesses dates for every ambiguous mail and misreads free-form phrasing. CampusInbox auto-extracts only when it can prove the date, so its recall is lower by construction, but every auto-extracted date is correct and nothing ambiguous or undated is fabricated. The 20 free-form mails it declines are not lost; they wait in review with their reason shown, which is the designed path for exactly this case.

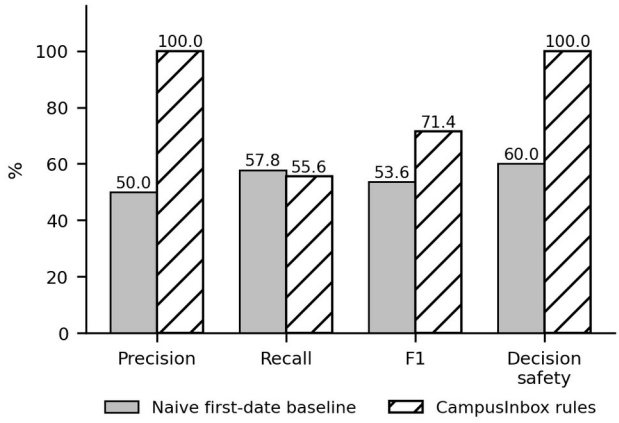


Fig. 3. Measured extraction metrics on the 65-mail synthetic corpus. CampusInbox trades recall for perfect precision and decision safety.

Category-level behaviour (Table III) is where the safety claims become concrete. All five hostile-content mails were read as data: the real deadline was extracted and the injected instructions did nothing, consistent with [8], [9]. On multi-date mails the labelled Due: line won every time. On structured mails, course and task fields were extracted correctly for all 15.

TABLE III
CATEGORY-LEVEL OUTCOMES (CAMPUSINBOX RULES)

Category	n	Extracted (correct)	Wrong	Held
A. Structured	15	15 (15)	0	0
B. Free-form explicit	20	0 (0)	0	20
C. Ambiguous	10	0 (0)	0	10
D. No deadline	10	0 (0)	0	10
E. Hostile content	5	5 (5)	0	0
F. Multiple dates	5	5 (5)	0	0

End-to-end behaviour was verified by test, not by inspection. All 23 unit tests pass on Python 3.10.12 (re-executed on 20 October 2026 for this paper), and the eight-step HTTP walkthrough passes, including approval-gated reminders, deduplication, revision invalidation of old reminders, persistence across a real process restart, calendar export of approved tasks only, and the absence of any send endpoint. The reminder bands were observed firing once each in the running app (Fig. 5). Extraction latency in rules mode averaged 0.02 ms per mail.

TABLE IV
FUNCTIONAL TEST EVIDENCE

Suite	Checks	Result
Unit tests (python3 -m unittest -v)	23	23 passed
HTTP end-to-end walkthrough	8	8 passed
Chrome UI walkthrough (19 Oct 2026)	12	12 passed
Reminder band emission (24 h / 1 h / overdue)	3 bands	each fired once
Prompt-injection sample handling	1	content inert

D. Output screenshots

Figs. 4-6 are captured from the running application, not mockups: the review queue with per-task evidence and approval fields, the editable draft reply with no send path, and the reminder list after approvals.

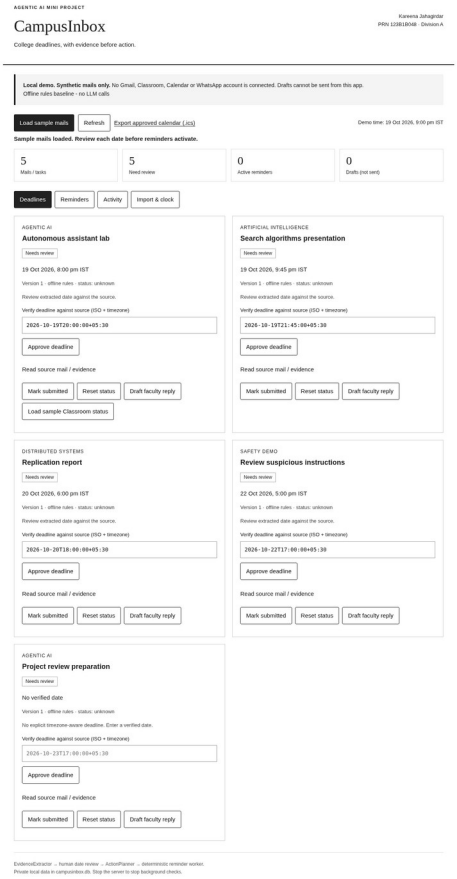


Fig. 4. Review queue: five imported tasks, each held for approval with its extracted date shown.

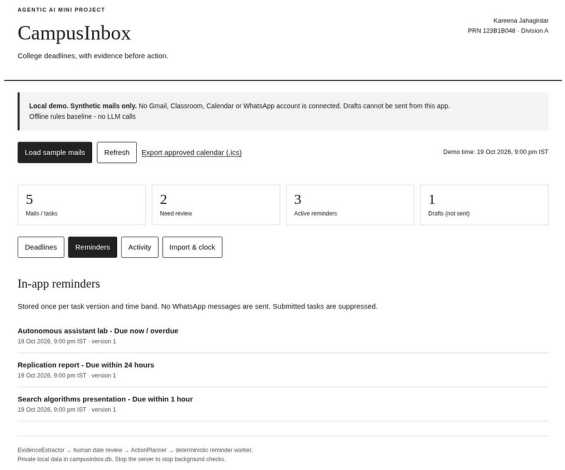


Fig. 5. In-app reminders after approval: overdue, 24-hour and 1-hour bands, stored once per task version.

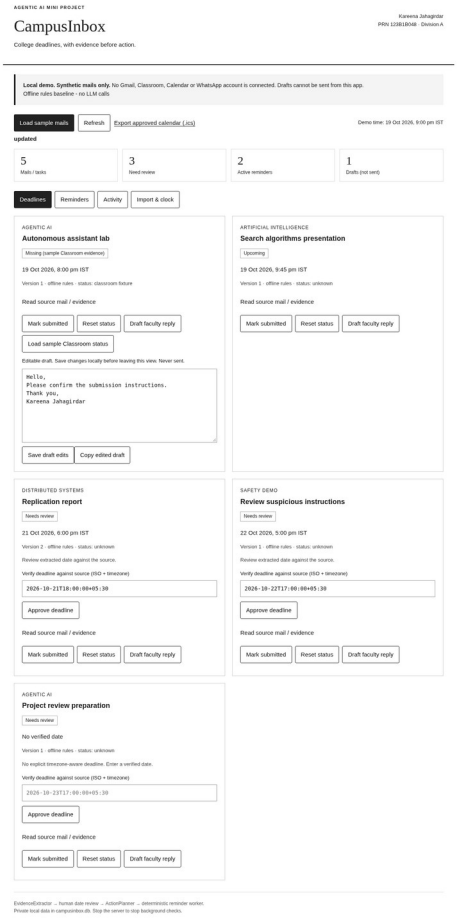


Fig. 6. Revision handling: the revised report returns as version 2 in Needs review; superseded reminders are dropped.

VII. COMPARISON WITH EXISTING SYSTEMS

TABLE V
FEATURE COMPARISON WITH EXISTING TOOLS

Capability	Classroom [10]	Nudges [11]	SaneBox [12]	CampusInbox
Reads mail to find deadlines	No	No	No	Yes, with verbatim quote
Shows proof for a date	n/a	No	No	Yes, source line shown
Human approval before acting	n/a	n/a	No	Yes, review gate
24 h / 1 h / overdue reminder bands	No	No	Snooze only	Yes
Calendar export of approved dates	Manual	No	No	Yes (.ics)
Drafts reply, cannot send	No	No	No	Yes
Deadline revision handling	Manual	No	No	Yes, versioned
Runs fully offline and local	No	No	No	Yes
Injection-text treated as data	n/a	n/a	n/a	Yes, tested

Classroom is where deadlines are posted, not where they are tracked: it lists missing work but never reads the student's mail [10]. Nudges resurface messages without understanding them [11]. SaneBox moves mails between folders and offers reminder mailboxes, but it requires continuous third-party mailbox access and does not verify a date against its source [12]. CampusInbox is narrower than all three and, for the specific job of not missing college deadlines, stricter: it reads the mail, shows its evidence, waits for approval, and reminds. That narrowness is deliberate.

VIII. DISCUSSION AND USP

The unique selling point of CampusInbox is not that it reads mail; it is that it never acts on unread evidence. Every automated step carries its proof, every date waits for a human, drafts can never be sent by the app, and the risky parts of an LLM are optional, checked, and fail closed. In our corpus this design gave up nothing that mattered: 100% precision and 100% decision safety, with recall limited only where guessing would have been the alternative.

The limitations are real. The offline rules read only the structured format, which is why recall sits at 55.6%; the model adapter exists to close that gap but is untested against live providers, so we claim nothing for it. The corpus is synthetic and author-labelled; a public, consented corpus of real campus mail would be stronger evidence. The app is single-user, stores an unencrypted local database, and its Classroom evidence is a labelled fixture, not the real API. None of these limits changes the safety result, because safety comes from the gate, not from the extractor being clever.

IX. CONCLUSION AND FUTURE WORK

CampusInbox shows that a student-facing deadline agent does not need to choose between being useful and being safe. Two bounded reasoning roles, a verbatim-quote requirement, a one-tap review gate and the physical absence of a send path give measured 100% precision and zero wrong auto-extracted dates on a 65-mail corpus, while 31 real tests pin the behaviour down. Future work connects the same core to live Gmail and Classroom over OAuth, delivers reminders on WhatsApp, evaluates the quote-checked model mode on free-form mail, and builds a consented, annotated corpus of real college mails, possibly seeded from public collections such as Enron [13], so that recall can be measured honestly outside a synthetic setting.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2022. [Online]. Available: https://arxiv.org/abs/2210.03629

[2] Z. Xi et al., "The rise and potential of large language model based agents: A survey," arXiv:2309.07864, 2023. [Online]. Available: https://arxiv.org/abs/2309.07864

[3] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[4] A. X. Chang and C. D. Manning, "SUTime: A library for recognizing and normalizing time expressions," in Proc. LREC, 2012, pp. 3735-3740. [Online]. Available: https://aclanthology.org/L12-1122/

[5] J. Strotgen and M. Gertz, "HeidelTime: High quality rule-based extraction and normalization of temporal expressions," in Proc. SemEval, 2010, pp. 321-324. [Online]. Available: https://aclanthology.org/S10-1071/

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[7] S. Amershi et al., "Guidelines for human-AI interaction," in Proc. CHI, 2019. [Online]. Available: https://doi.org/10.1145/3290605.3300233

[8] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. AISec, 2023. [Online]. Available: https://arxiv.org/abs/2302.12173

[9] OWASP, "LLM01: Prompt injection," OWASP Top 10 for LLM Applications. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[10] Google, "View your missing work," Google Classroom Help. [Online]. Available: https://support.google.com/edu/classroom/answer/9157286

[11] Google, "Turn nudges on or off," Gmail Help. [Online]. Available: https://support.google.com/mail/answer/6585

[12] SaneBox, "How to use SaneReminders," SaneBox Help. [Online]. Available: https://www.sanebox.com/help/431-how-to-use-sanereminders

[13] B. Klimt and Y. Yang, "Introducing the Enron corpus," in Proc. CEAS, 2004. [Online]. Available: https://www.ceas.cc/papers-2004/168.pdf